

Лабораторная работа № 1

Тема: Подключение Internet Shop API к PostgreSQL

Цель работы: Создать базу данных PostgreSQL для интернет-магазина, подключить её к существующему Express-приложению через пакет pg и заменить чтение товаров из массива операциями SELECT.

В этой лабораторной работе реализуем только **Read**:

GET /api/products

GET /api/products/:id

INSERT, UPDATE и DELETE из Node.js будут подробно реализованы в следующей лекции.

1. Проверьте PostgreSQL

Убедитесь, что PostgreSQL Server запущен.

Для работы можно использовать:

- pgAdmin;
- psql;
- другой SQL-клиент.

2. Создайте пользователя приложения

Подключитесь к PostgreSQL с административными правами и выполните:

```
CREATE USER shop_user  
WITH PASSWORD 'college123';
```

Пароль college123 используется только для учебной локальной среды.

3. Создайте базу данных

Выполните:

```
CREATE DATABASE shop_db  
OWNER shop_user;
```

В результате должна появиться:

shop_db

Владельцем базы является:

shop_user

4. Подключитесь к shop_db как shop_user

В pgAdmin можно создать отдельное подключение к серверу с параметрами:

Host: localhost

Port: 5432

Database: shop_db

User: shop_user

Password: college123

Либо используйте psql, если он доступен:
psql -h localhost -U shop_user -d shop_db

5. Создайте таблицу products

Выполните в базе shop_db:

```
CREATE TABLE products (  
  id INTEGER  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  
  name VARCHAR(150)  
    NOT NULL,  
  
  price INTEGER  
    NOT NULL  
    CHECK (price > 0),  
  
  category VARCHAR(100)  
    NOT NULL,  
  
  in_stock BOOLEAN  
    NOT NULL  
    DEFAULT true  
);
```

6. Добавьте тестовые данные

```
INSERT INTO products  
(  
  name,  
  price,  
  category,  
  in_stock  
)  
VALUES  
(  
  'Keyboard',  
  15000,  
  'Accessories',  
  true  
)  
(  
  'Mouse',  
  8000,  
  'Accessories',
```

```
    true
  ),
  (
    'Monitor',
    85000,
    'Monitors',
    false
  );
```

7. Проверьте таблицу

```
SELECT *
FROM products
ORDER BY id;
```

Должны появиться три товара.

Пример:

```
1 | Keyboard | 15000 | Accessories | true
2 | Mouse   | 8000   | Accessories | true
3 | Monitor  | 85000  | Monitors   | false
```

8. Откройте Express-проект

Продолжаем существующий проект:

express-shop/

Пример текущей структуры:

express-shop/

```
|— controllers/
|   |— productController.js
|— errors/
|— middleware/
|— routes/
|   |— productRoutes.js
|— services/
|   |— productService.js
|— validators/
|— public/
|— app.js
|— package.json
|— package-lock.json
```

9. Установите PostgreSQL-драйвер

В терминале проекта:

```
npm install pg
```

node-postgres устанавливается пакетом pg, поддерживает Promise/async/await и предоставляет встроенный пул соединений.

10. Создайте каталог config

Итог:

```
express-shop/  
├── config/  
│   └── db.js  
├── controllers/  
├── routes/  
├── services/  
├── app.js  
└── package.json
```

11. Создайте .env

В корне проекта:

PGHOST=localhost

PGPORT=5432

PGDATABASE=shop_db

PGUSER=shop_user

PGPASSWORD=college123

Пакет pg использует стандартные параметры подключения

PostgreSQL: host, port, database, user и password; он также

поддерживает соответствующие переменные окружения PGHOST, PGPORT, PGDATABASE, PGUSER и PGPASSWORD.

12. Добавьте .env в .gitignore

Создайте или дополните:

node_modules/

.env

Пароль базы данных не должен попадать в Git-репозиторий.

13. Создайте config/db.js

```
import {  
  loadEnvFile  
} from "node:process";
```

```
import {  
  Pool  
} from "pg";
```

```
loadEnvFile();
```

```
export const pool = new Pool({  
  host:  
    process.env.PGHOST,
```

```
port:
  Number(
    process.env.PGPORT
  ),

database:
  process.env.PGDATABASE,

user:
  process.env.PGUSER,

password:
  process.env.PGPASSWORD
});
```

`process.loadEnvFile()` загружает `.env` в `process.env`, поэтому дополнительный пакет `dotenv` для этой конфигурации не требуется в используемой современной версии `Node.js`.

14. Создайте тест подключения

В корне проекта создайте:

`test-db.js`

Код:

```
import {
  pool
} from "../config/db.js";

try {
  const result =
    await pool.query(
      "SELECT NOW() AS current_time"
    );

  console.log(
    "PostgreSQL connected:"
  );

  console.log(
    result.rows[0]
  );
} catch (error) {
  console.error(
    "Database connection error:"
  );
}
```

```
    console.error(  
      error.message  
    );  
  } finally {  
    await pool.end();  
  }  
}
```

15. Запустите проверку

node test-db.js

Ожидается сообщение вида:

PostgreSQL connected:

```
{ current_time: ... }
```

Если возникла ошибка, проверьте:

PGHOST

PGPORT

PGDATABASE

PGUSER

PGPASSWORD

PostgreSQL использует клиент-серверную модель: приложение подключается к серверу базы данных, передаёт запрос, а сервер выполняет его и возвращает результат.

16. Удалите временный тест

После успешной проверки файл:

test-db.js

можно удалить.

pool.end() в основном Express-приложении после каждого запроса вызывать **не нужно** — Pool должен оставаться доступным следующим HTTP-запросам.

17. Откройте productService.js

До подключения PostgreSQL там использовался массив:

```
let products = [  
  // ...  
];
```

и операции:

```
products.find();
```

```
products.push();
```

```
products.splice();
```

Теперь источником данных станет PostgreSQL.

18. Импортируйте pool

В начале productService.js:

```
import {
```

```
pool
} from "../config/db.js";
```

19. Перепишите getAllProducts()

```
export async function getAllProducts() {
  const result =
    await pool.query(
      `SELECT
        id,
        name,
        price,
        category,
        in_stock AS "inStock"
      FROM products
      ORDER BY id`
    );

  return result.rows;
}
```

Обратите внимание на:

in_stock AS "inStock"

В PostgreSQL используется имя:

in_stock

а REST API продолжает возвращать:

inStock

Таким образом менять frontend не требуется.

20. Перепишите findProductById()

```
export async function findProductById(
  id
) {
  const result =
    await pool.query(
      `SELECT
        id,
        name,
        price,
        category,
        in_stock AS "inStock"
      FROM products
      WHERE id = $1`,
      [id]
    );
}
```

```
    return result.rows[0];  
  }  
}
```

21. Обратите внимание на \$1

Не делайте так:

```
const sql =  
  "SELECT * FROM products WHERE id = " +  
  id;
```

Используйте:

```
await pool.query(  
  "SELECT * FROM products WHERE id = $1",  
  [id]  
);
```

Значение пользователя передаётся отдельно от текста SQL.

22. Измените getProducts в Controller

Поскольку Service теперь обращается к базе асинхронно:

```
export async function getProducts(  
  req,  
  res,  
  next  
) {  
  try {  
    const products =  
      await getAllProducts();  
  
    res.json(products);  
  } catch (error) {  
    next(error);  
  }  
}
```

23. Измените getProductById

```
export async function getProductById(  
  req,  
  res,  
  next  
) {  
  try {  
    const id =  
      Number(req.params.id);  
  
    const product =  
      await findProductById(id);
```

```
    if (!product) {  
      return next(  
        new AppError(  
          "Product not found",  
          404  
        )  
      );  
    }  
  
    res.json(product);  
  } catch (error) {  
    next(error);  
  }  
}
```

24. Запустите сервер

npm start

или вашей текущей командой запуска проекта.

25. Проверьте получение всех товаров

GET /api/products

Ожидаемый результат:

```
[  
  {  
    "id": 1,  
    "name": "Keyboard",  
    "price": 15000,  
    "category": "Accessories",  
    "inStock": true  
  },  
  {  
    "id": 2,  
    "name": "Mouse",  
    "price": 8000,  
    "category": "Accessories",  
    "inStock": true  
  }  
]
```

26. Проверьте получение одного товара

GET /api/products/2

Ожидается:

```
{
```

```
"id": 2,  
"name": "Mouse",  
"price": 8000,  
"category": "Accessories",  
"inStock": true  
}
```

27. Проверьте отсутствующий товар

GET /api/products/999

Ожидается:

404 Not Found

и существующий обработчик ошибок:

```
{  
  "message": "Product not found"  
}
```

28. Проверьте постоянное хранение

Через pgAdmin выполните:

INSERT INTO products

```
(  
  name,  
  price,  
  category,  
  in_stock  
)  
VALUES  
(  
  'SSD 2TB',  
  65000,  
  'Components',  
  true  
);
```

Затем снова:

GET /api/products

Новый товар должен появиться в REST API.

29. Перезапустите Node.js

Остановите Express-сервер:

Ctrl + C

Запустите снова.

Повторите:

GET /api/products

Товар SSD 2TB должен остаться.

Это главное отличие от прежнего массива products[].

30. Проведите эксперимент с ошибкой подключения

Временно измените:

PGPORT=5433

Перезапустите сервер и выполните запрос.

Приложение должно получить ошибку подключения к базе, а ваш централизованный errorHandler — обработать переданную ошибку.

После эксперимента верните:

PGPORT=5432

Итоговая структура проекта

```
express-shop/
├── config/
│   └── db.js
├── controllers/
│   └── productController.js
├── errors/
├── middleware/
├── routes/
│   └── productRoutes.js
├── services/
│   └── productService.js
├── validators/
├── public/
├── .env
├── .gitignore
├── app.js
├── package.json
└── package-lock.json
```

Требования к результату

Приложение должно:

- использовать PostgreSQL вместо массива для чтения товаров;
- содержать базу shop_db;
- содержать таблицу products;
- подключаться к PostgreSQL через пакет pg;
- использовать Pool;
- хранить параметры подключения в .env;
- иметь отдельный config/db.js;
- выполнять SELECT через pool.query();
- получать строки через result.rows;
- использовать параметр \$1 при поиске по id;
- возвращать прежний формат REST API;
- возвращать 404 для отсутствующего товара;

- передавать ошибки PostgreSQL в существующий errorHandler;
- сохранять данные после перезапуска Node.js.